

Sprawozdanie z realizacji projektu

System rezerwacji wizyt lekarskich — ClearBook

Baza danych: PostgreSQL · **Backend:** Spring Boot 4.0.5 / Java 25 · **Frontend:** Next.js 15 / React 19

1. Wykorzystywane technologie

1.1 Warstwa bazy danych

PostgreSQL 16 — relacyjna baza danych wybrana z uwagi na rozbudowane wsparcie dla blokad pesymistycznych (**SELECT FOR UPDATE**), sprawdzonych mechanizmów transakcyjnych zgodnych z ACID, natywnego typu **UUID** jako klucza głównego oraz operatora **COALESCE** i funkcji agregujących używanych w zapytaniach raportujących.

Flyway 10 — narzędzie do wersjonowania schematu bazy danych. Migracje wykonywane są automatycznie przy starcie aplikacji w kolejności numerycznej (**V1**, **V2**, **V3**, **V4**). Podejście to gwarantuje powtarzalność struktury bazy na każdym środowisku i pełną historię zmian schematu w repozytorium.

1.2 Warstwa backendu

Technologia	Wersja	Rola
Java	25	Język implementacji backendu
Spring Boot	4.0.5	Framework aplikacyjny (IoC, auto-konfiguracja)
Spring Data JPA / Hibernate	7.x	ORM — mapowanie encji, zarządzanie transakcjami, JPQL
Spring Security	7.x	Uwierzytelnianie JWT, autoryzacja metodami (@PreAuthorize)
Spring WebSocket / STOMP	—	Powiadomienia w czasie rzeczywistym
Lombok	1.18.x	Redukcja kodu szablonowego (getterzy, builderzy, konstruktory)
jjwt	0.12.x	Generowanie i weryfikacja tokenów JWT
springdoc-openapi	2.x	Automatyczna dokumentacja API (Swagger UI)
Testcontainers	1.20.x	Integracyjne środowisko testowe z prawdziwą instancją PostgreSQL

1.3 Warstwa frontendu

Next.js 15 (React 19) — framework SSR/RSC. Warstwa serwerowa pełni rolę proxy między przeglądarką a backendem Spring, eliminując konieczność CORS i chroniąc tokeny JWT przed ekspozycją przez

httpOnly cookies. Na frontencie zaimplementowano pełny interfejs dla czterech ról użytkownika.

2. Model danych

2.1 Diagram encji (opis słowny)

Model danych składa się z **14 tabel** podzielonych na pięć domen:



Wszystkie klucze główne są typu **UUID** generowanego przez bazę danych (**GenerationType.UUID**), co eliminuje ryzyko kolizji w środowiskach rozproszonych i zapobiega przewidywalności identyfikatorów przez klientów.

2.2 Opis tabel

Domena użytkowników i tożsamości

users — centralna tabela dla wszystkich typów kont. Pole **role** przyjmuje wartości **USER**, **DOCTOR**, **MANAGER**, **ADMIN** ze sprawdzaniem na poziomie bazy (**CHECK** constraint). Pole **status** modelu **UNVERIFIED** → **PENDING** → **ACTIVE** odzwierciedla wieloetapowy przepływ aktywacji konta. Tabela implementuje interfejs **UserDetails** wymagany przez Spring Security.

verification_tokens, **password_reset_tokens** — jednorazowe tokeny z terminem ważności (**expiry_date**) wysyłane email. Klucz obcy **user_id** ma ograniczenie **UNIQUE** — każdy użytkownik może mieć co najwyżej jeden aktywny token danego rodzaju.

refresh_tokens — tokeny odświeżania JWT z polem **revoked** umożliwiającym wykrywanie kradzieży sesji. Każde użycie tokenu generuje nowy (rotacja), a prezentacja już unieważnionego tokenu powoduje natychmiastowe unieważnienie wszystkich sesji danego użytkownika.

Domena profili lekarzy i placówek

doctor_profiles — rozszerzenie konta lekarza o dane publiczne: **public_id** (czytelny identyfikator URL), **bio**, **average_rating**, **total_reviews**, **verification_status**. Pole **is_public** steruje widocznością w wyszukiwarce.

doctor_profile_specializations — tabela łącząca (M:N) między **doctor_profiles** a **specializations**. Klucz złożony (**doctor_profile_id**, **specialization_id**) zapobiega duplikatom.

specializations — słownik 22 specjalizacji medycznych seedowanych przy starcie aplikacji (**CardioLog**y, **Neuro**logy, itd.).

medical_centers — placówki medyczne z cyklem życia **PENDING_APPROVAL** → **ACTIVE** / **SUSPENDED**. Nowo zarejestrowane centrum wymaga zatwierdzenia przez administratora platformy.

center_memberships — relacja M:N między lekarzami a placówkami z rozbudowaną semantyką: rola (**ADMIN/MEMBER**), status (**INVITED/ACTIVE/SUSPENDED/REJECTED**), audyt (**invited_by**, **invited_at**, **joined_at**). Ograniczenie **UNIQUE(user_id, center_id)** gwarantuje unikalność członkostwa.

invite_codes — kody zaproszeń w formacie **CB-XXXX-XXXX** generowane przy użyciu kryptograficznie bezpiecznego generatora (**SecureRandom**). Lekarze udostępniają kody administratorom centrów w celu dołączenia.

Domena harmonogramu i wizyt

availability_blocks — bloki dyspozycyjności lekarza w danej placówce (np. „wtorek 08:00–16:00 w Centrum Zdrowia”). Pole **is_deleted** umożliwia miękkie usuwanie bez naruszania integralności referencyjnej istniejących wizyt.

doctor_services — katalog usług lekarza z ceną i czasem trwania. Pole **active** umożliwia dezaktywację usług z zachowaniem historycznych danych wizyt (wzorzec Copy-on-Write przy aktualizacji — opisany w sekcji 3).

appointments — centralna tabela wizyt. Pole **status** (**RESERVED/SCHEDULED/COMPLETED/CANCELLED/NO_SHOW**) odzwierciedla pełny cykl życia wizyty. Pole **reserved_until** implementuje mechanizm 15-minutowej rezerwacji wstępnej ze automatycznym wygaśnięciem.

appointment_reviews — relacja 1:1 z **appointments** (każda wizyta może mieć co najwyżej jedną recenzję). Zawiera ocenę liczbową, komentarz pacjenta i możliwość odpowiedzi lekarza.

notifications (dodane w V4) — skrzynka odbiorcza powiadomień in-app z polem **is_read**. Indeks częściowy **WHERE is_read = false** przyspiesza zapytania o liczbę nieprzeczytanych.

2.3 Migracje Flyway

```
V1 – schemat bazowy (wszystkie tabele, klucze obce, constraints)
V2 – tabela refresh_tokens (obsługa rotacji tokenów)
V3 – kolumna is_deleted w availability_blocks (miękkie usuwanie)
V4 – kolumny license_file_path, verification_status, reminder_sent;
    tabela notifications; 15 indeksów wydajnościowych na kolumnach FK
```

Konfiguracja **baseline-version=0** oznacza, że na nowej bazie wykonywane są wszystkie migracje od V1.

3. Realizacja operacji w bazie danych

3.1 Operacje CRUD

Operacje CRUD zrealizowano dla wszystkich głównych encji przy użyciu Spring Data JPA. Dostęp do danych odbywa się przez interfejsy dziedziczące po `JpaRepository<T, UUID>`, co eliminuje boilerplate SQL dla standardowych operacji.

Przykład — wyszukiwanie lekarzy z filtrami (`DoctorProfileRepository`):

```
@Query("SELECT p FROM DoctorProfile p " +
        "JOIN FETCH p.specializations s " +
        "JOIN p.user u " +
        "LEFT JOIN CenterMembership cm ON cm.user = u AND cm.status =
:active " +
        "WHERE p.isPublic = true " +
        "AND (:spec IS NULL OR s.code = :spec) " +
        "AND (:city IS NULL OR LOWER(cm.center.city) = LOWER(:city))")
Page<DoctorProfile> searchPublic(
    @Param("spec") String specialization,
    @Param("city") String city,
    @Param("active") MembershipStatus active,
    Pageable pageable
);
```

Zapytanie łączy cztery tabele (`doctor_profiles`, `specializations`, `users`, `center_memberships`), stosuje `JOIN FETCH` dla eliminacji problemu N+1 i działa na danych paginowanych. Globalny limit strony `spring.data.web.pageable.max-page-size=100` zabezpiecza przed żądaniami `?size=10000`.

Wzorzec Copy-on-Write przy aktualizacji usług lekarza (`DoctorServiceManager`):

Aktualizacja usługi, do której już istnieją zarezerwowane wizyty, nie nadpisuje rekordu. Stary rekord jest dezaktywowany (`active=false`), a nowy tworzony z nowym UUID. Dzięki temu historyczne wizyty zachowują prawidłowe referencje do pierwotnych danych usługi.

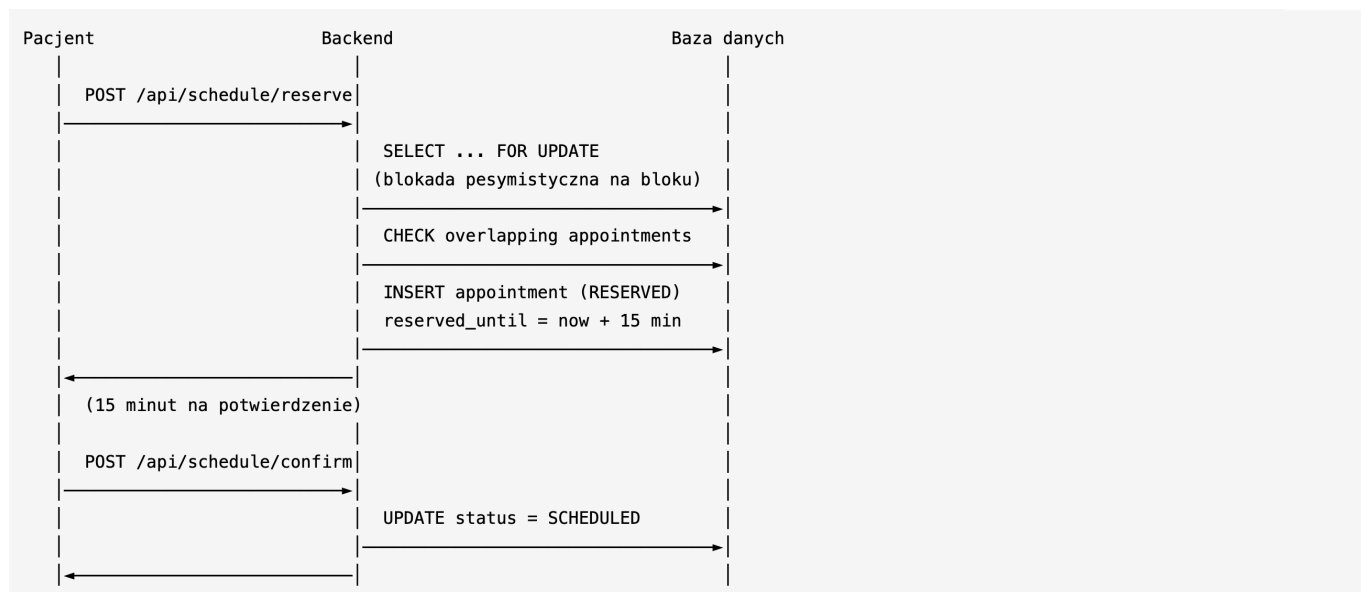
```
@Transactional
public DoctorServiceResponse updateService(User doctor, UUID serviceId,
...) {
    DoctorService existing = /* ... */;
    if (appointmentRepository.existsByService(existing)) {
        // Copy-on-Write: deactivate old, create new
        existing.setActive(false);
        doctorServiceRepository.save(existing);
        return toResponse(doctorServiceRepository.save(/* new service */));
    }
    // In-place update – no appointments reference this service
    existing.setName(request.getName());
    /* ... */
}
```

```
    return toResponse(doctorServiceRepository.save(existing));
}
```

3.2 Operacje transakcyjne — rezerwacja wizyt z kontrolą współbieżności

Jest to kluczowa operacja demonstrująca kontrolę równoczesnego dostępu do ograniczonego zasobu (sloty czasowe lekarza).

Przepływ rezerwacji (2-etapowy)



Implementacja blokady pesymistycznej

```
// AvailabilityBlockRepository.java
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT b FROM AvailabilityBlock b WHERE b.id = :id AND b.isDeleted = false")
Optional<AvailabilityBlock> findByIdWithPessimisticLock(@Param("id") UUID id);
```

Adnotacja `@Lock(LockModeType.PESSIMISTIC_WRITE)` generuje klauzulę `SELECT ... FOR UPDATE` w PostgreSQL. Każda transakcja rezerwacji blokuje wiersz bloku dostępności, uniemożliwiając innym transakcjom równoczesną modyfikację. Konkurencyjne transakcje są kolejgowane przez bazę danych i wykonywane sekwencyjnie.

Zapobieganie podwójnej rezerwacji

Sama blokada pesymistyczna gwarantuje wyłączość, ale nie sprawdza stanu slotu. Zastosowano dodatkowe zapytanie walidacyjne:

```
// AppointmentRepository.java
@Query("SELECT COUNT(a) > 0 FROM Appointment a " +
```

```

        "WHERE a.block = :block " +
        "AND (a.status IN ('SCHEDULED', 'COMPLETED') " +
        "    OR (a.status = 'RESERVED' AND a.reservedUntil >
CURRENT_TIMESTAMP)) " +
        "AND (a.startTime < :endTime AND a.endTime > :startTime)")
boolean existsOverlappingAppointment(
    @Param("block") AvailabilityBlock block,
    @Param("startTime") LocalDateTime startTime,
    @Param("endTime") LocalDateTime endTime
);

```

Zapytanie stosuje jawne listy dozwolonych statusów (whitelist) zamiast wykluczeń (blacklist), co chroni przed błędami przy dodawaniu nowych statusów w przyszłości.

Automatyczne zwalnianie przeterminowanych rezerwacji

Wizyty w statusie **RESERVED**, których czas `reserved_until` minął, są traktowane jako wolne w obu mechanizmach (wyszukiwanie dostępnych slotów oraz walidacja kolizji) — filtry pomijają je w czasie rzeczywistym. Zadanie harmonogramowe (`ExpiredReservationCleanupTask`) czyści fizycznie takie rekordy co 5 minut:

```

@Scheduled(fixedRate = 5 * 60 * 1000)
@Transactional
public void cleanupExpiredReservations() {
    int cancelled = appointmentRepository.cancelExpiredReservations();
}

// AppointmentRepository.java
@Modifying
@Query("UPDATE Appointment a SET a.status = 'CANCELLED' " +
        "WHERE a.status = 'RESERVED' AND a.reservedUntil <
CURRENT_TIMESTAMP")
int cancelExpiredReservations();

```

Rotacja tokenów odświeżania z detekcją kradzieży

Drugim przykładem złożonej operacji transakcyjnej jest mechanizm bezpiecznych sesji. Każde użycie refresh tokenu natychmiast go unieważnia i wydaje nowy (rotacja). Prezentacja już unieważnionego tokenu jest sygnałem potencjalnej kradzieży sesji — wszystkie aktywne sesje użytkownika są natychmiast unieważniane:

```

@Transactional(noRollbackFor = SecurityException.class)
public RefreshToken rotate(String tokenValue) {
    RefreshToken token = refreshTokenRepository.findByToken(tokenValue)
        .orElseThrow(() -> new IllegalArgumentException("Token not
found."));

    if (token.isRevoked()) {

```

```
// Reuse detected – wipe all sessions for this user
refreshTokenRepository.revokeAllUserTokens(token.getUser());
throw new SecurityException("Token reuse detected. All sessions
revoked.");
}
token.setRevoked(true);
refreshTokenRepository.save(token);
return create(token.getUser()); // issue new token
}
```

Adnotacja `noRollbackFor = SecurityException.class` gwarantuje, że unieważnienie wszystkich tokenów (operacja wykonana przed rzuceniem wyjątku) zostanie zatwierdzone w bazie nawet mimo propagacji wyjątku.

3.3 Operacje raportujące

Moduł raportowania (`ReportService`) generuje miesięczne statystyki lekarza przy użyciu czterech zapytań agregujących wykonywanych w ramach jednej transakcji tylko do odczytu

`@Transactional(readOnly = true)`:

Rozkład statusów wizyt:

```
@Query("SELECT a.status, COUNT(a) FROM Appointment a " +
        "WHERE a.block.doctor = :doctor " +
        "AND a.startTime >= :from AND a.startTime < :to " +
        "GROUP BY a.status")
List<Object[]> countByStatusForDoctor(/* ... */);
```

Suma przychodów z zrealizowanych wizyt:

```
@Query("SELECT COALESCE(SUM(a.service.price), 0) FROM Appointment a " +
        "WHERE a.block.doctor = :doctor AND a.status = 'COMPLETED' " +
        "AND a.startTime >= :from AND a.startTime < :to")
BigDecimal sumEarningsForDoctor(/* ... */);
```

Top 5 najpopularniejszych usług:

```
@Query("SELECT a.service.name, COUNT(a) FROM Appointment a " +
        "WHERE a.block.doctor = :doctor AND a.status = 'COMPLETED' " +
        "AND a.startTime >= :from AND a.startTime < :to " +
        "GROUP BY a.service.name ORDER BY COUNT(a) DESC")
List<Object[]> topServicesByDoctor(/* ... */, Pageable pageable);
```

Rozkład wizyt wg placówek (nazwa, łączna liczba, zrealizowane, przychód):

```
@Query("SELECT b.center.name, COUNT(a), " +
    "SUM(CASE WHEN a.status = 'COMPLETED' THEN 1 ELSE 0 END), " +
    "COALESCE(SUM(CASE WHEN a.status = 'COMPLETED' THEN a.service.price
ELSE 0 END), 0) " +
    "FROM Appointment a JOIN a.block b " +
    "WHERE b.doctor = :doctor " +
    "AND a.startTime >= :from AND a.startTime < :to " +
    "GROUP BY b.center.name ORDER BY COUNT(a) DESC")
List<Object[]> perCenterBreakdownForDoctor(/* ... */);
```

Ostatnie zapytanie łączy trzy tabele (`appointments`, `availability_blocks`, `medical_centers`) z warunkową agregacją (`CASE WHEN`) w jednym przebiegu bazy danych, eliminując konieczność wielokrotnych zapytań po stronie aplikacji.

Aktualizacja ocen lekarza następuje zdarzeniowo — po każdej zmianie recenzji publikowane jest zdarzenie `ReviewChangedEvent`, które jest obsługiwane po zatwierdzeniu transakcji (`@TransactionalEventListener(AFTER_COMMIT)`). Listener przelicza średnią ocenę i aktualizuje denormalizowane pola `average_rating` i `total_reviews` w `doctor_profiles`:

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
@Async
public void onReviewChanged(ReviewChangedEvent event) {
    // Recalculate from DB and update doctor_profiles
}
```

4. Demonstracja możliwości zastosowanych technologii

4.1 Spring Data JPA — eliminacja problemu N+1

Standardowe zapytanie pobierające listę wizyt pacjenta ładowałoby każdą wizytę osobno, a następnie dla każdej osobno pobierało lekarza, profil lekarza, centrum medyczne i usługę (N+1 dodatkowych zapytań). Zastosowane `JOIN FETCH` spłaszcza to do jednego zapytania SQL:

```
@Query(value = "SELECT a FROM Appointment a " +
    "JOIN FETCH a.block b " +
    "JOIN FETCH b.doctor d " +
    "LEFT JOIN FETCH d.doctorProfile " +
    "JOIN FETCH b.center " +
    "JOIN FETCH a.service " +
    "WHERE a.patient = :patient ...",
    countQuery = "SELECT COUNT(a) FROM Appointment a WHERE a.patient = :patient ...")
Page<Appointment> findByPatient(@Param("patient") User patient, ...);
```

Oddzielne `countQuery` jest wymagane przy paginacji z `JOIN FETCH` — bez niego Hibernate generuje nieoptymalne zapytanie liczące rekordy z wszystkimi złączeniami.

Dla przypadków masowego ładowania powiązanych encji (np. mapowanie listy 50 lekarzy i ich profili) zastosowano metodę `findProfileMapByUsers` zwracającą mapę `Map<UUID, DoctorProfile>` w jednym zapytaniu, oraz właściwość `hibernate.default_batch_fetch_size=100` ograniczającą liczbę zapytań wsadowych.

4.2 Flyway — zarządzanie schematem w czasie

Flyway zastąpił tryb `spring.jpa.hibernate.ddl-auto=update`, który w środowisku produkcyjnym jest niezalecany (nie usuwa kolumn, nie zarządza indeksami). Tryb `validate` gwarantuje, że aplikacja nie uruchomi się gdy schemat bazy nie zgadza się z mapowaniem JPA — co stanowi jawne potwierdzenie spójności zamiast cichego obejścia.

Każda migracja jest niemodyfikowalna po wdrożeniu (Flyway weryfikuje sumy kontrolne). Nowe zmiany wymagają nowego pliku migracji — co wymusza pełną historię zmian w repozytorium.

4.3 Architektura zdarzeń (Spring Application Events)

Powiadomienia email i aktualizacje statystyk odsprzęgnięto od głównych transakcji przez mechanizm zdarzeń aplikacyjnych:

```
// W AppointmentService.confirmAppointment()
eventPublisher.publishEvent(new NotificationEvent(doctor, "New
appointment", message));

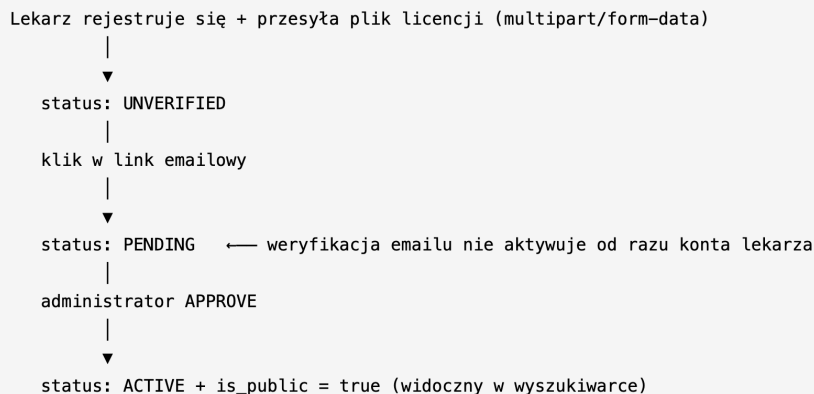
// W listener — wykonuje się AFTER_COMMIT, poza transakcją biznesową
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
@Async
public void onNotification(NotificationEvent event) {
    notificationService.create(event.getUser(), event.getTitle(),
    event.getMessage());
    // Wysyłka WebSocket i email
}
```

Adnotacja `AFTER_COMMIT` gwarantuje, że powiadomienie zostanie wysłane tylko jeśli transakcja główna zakończyła się sukcesem — eliminując scenariusz, w którym pacjent otrzymuje email o rezerwacji, a wizyta finalnie nie istnieje w bazie.

4.4 WebSocket / STOMP — powiadomienia w czasie rzeczywistym

Lekarze i pacjenci otrzymują powiadomienia push (nowa wizyta, anulowanie, recenzja) przez trwałe połączenie WebSocket z protokołem STOMP. Każdy użytkownik subskrybuje swój prywatny kanał `/user/queue/notifications`. Uwierzytelnianie przez JWT odbywa się przy nawiązywaniu połączenia (STOMP `CONNECT`) z dodatkową weryfikacją statusu konta przy kolejnych komendach.

4.5 Wieloetapowy przepływ aktywacji konta



Pacjenci i managerowie przechodzą bezpośrednio **UNVERIFIED** → **ACTIVE** po weryfikacji emailu. Lekarze wymagają dodatkowego zatwierdzenia przez administratora platformy.

5. Testy integracyjne

Testy realizowano przy użyciu **Testcontainers** — biblioteki uruchamiającej prawdziwą instancję PostgreSQL (obraz **postgres:16-alpine**) w kontenerze Docker na czas wykonania testów. Każdy test jest izolowany przez rollback transakcji (**@Transactional** na klasie bazowej **AbstractIntegrationTest**).

Wysyłka emaili jest mockowana (**@MockitoBean JavaMailSender**) — **EmailService** wykonuje się normalnie, ale wywołanie **send()** jest no-op.

Pokrycie testami

Plik testu	Testowane operacje
AppointmentConcurrencyIntegrationTest	Blokada pesymistyczna — 10 wątków vs. 1 slot
AppointmentServiceIntegrationTest	Pełny cykl wizyty: reserve → confirm → complete/cancel
AvailabilityServiceIntegrationTest	Tworzenie/usuwanie bloków, overlapping validation, Copy-Week
DoctorServiceManagerIntegrationTest	CRUD usług, Copy-on-Write przy aktualizacji
ReviewServiceIntegrationTest	Tworzenie recenzji, odpowiedzi, aktualizacja statystyk
MedicalCenterServiceIntegrationTest	Tworzenie centrum, zapraszanie lekarzy, zarządzanie składem
DoctorProfileServiceIntegrationTest	Tworzenie/aktualizacja profilu, wyszukiwanie
RefreshTokenServiceIntegrationTest	Rotacja tokenów, wykrywanie ponownego użycia
InviteCodeServiceIntegrationTest	Generowanie, odnawianie, rozwiązywanie kodu

Test współbieżności

```
@Test
@DisplayName("Should allow only one reservation under concurrent access by
10 patients")
void shouldPreventDoubleBookingUnderConcurrentLoad() {
    int threadCount = 10;
    ExecutorService executorService =
        Executors.newFixedThreadPool(threadCount);

    // 10 pacjentów próbuje zarezerwować ten sam slot jednocześnie
    List<Callable<Boolean>> tasks = concurrentPatients.stream()
        .map(patient -> (Callable<Boolean>) () -> {
            try {
                appointmentService.reserveSlot(patient, request);
                return true;
            } catch (Exception e) {
                return false;
            }
        }).toList();

    List<Future<Boolean>> results = executorService.invokeAll(tasks);

    // Tylko jedna rezerwacja powinna się udać
    assertThat(successfulReservations).isEqualTo(1);
    assertThat(failedReservations).isEqualTo(threadCount - 1);
}
```

Test potwierdza, że blokada pesymistyczna działa poprawnie w warunkach równoległego dostępu — dokładnie jedna rezerwacja jest zatwierdzana, pozostałe 9 otrzymuje błąd.

6. Instrukcja uruchomienia

6.1 Wymagania

- **Docker Desktop** (do uruchomienia PostgreSQL)
- **Java 25** (JDK)
- **Node.js 22+** i **npm**
- **Maven** (lub użyć dołączonego `./mvnw`)

6.2 Uruchomienie bazy danych

```
docker run --name clearbook-db \
-e POSTGRES_DB=clearbook \
-e POSTGRES_USER=clearbook_user \
-e POSTGRES_PASSWORD=clearbook_pass \
-p 5432:5432 \
-d postgres:16-alpine
```

6.3 Konfiguracja backendu

Należy utworzyć plik `api/.env` na podstawie poniższego szablonu:

```
DB_NAME=clearbook
DB_USER=clearbook_user
DB_PASSWORD=clearbook_pass

# Wygenerować losowy sekret base64 (min. 256 bitów)
# np.: openssl rand -base64 64
JWT_SECRET=<base64-secret>

# Opcjonalne – wartości domyślne jak poniżej
JWT_ACCESS_EXPIRATION_MS=900000
JWT_REFRESH_EXPIRATION_DAYS=7

# Konfiguracja Gmail SMTP (do wysyłki emaili weryfikacyjnych)
EMAIL_USERNAME=<adres-gmail>
EMAIL_2FA_PASSWORD=<hasło-aplikacji-google>

FRONTEND_URL=http://localhost:3000
```

6.4 Uruchomienie backendu

```
cd api
./mvnw spring-boot:run
```

Flyway automatycznie wykona migracje V1–V4 przy pierwszym starcie. Aplikacja uruchomi się na porcie **8080**.

Dokumentacja API (Swagger UI) dostępna pod adresem:

<http://localhost:8080/swagger-ui/index.html>

6.5 Uruchomienie frontendu

```
cd web
npm install
```

Należy utworzyć plik `web/.env.local`:

```
SPRING_API_URL=http://localhost:8080
NEXT_PUBLIC_WS_URL=ws://localhost:8080/ws
```

Następnie:

```
npm run dev
```

Aplikacja dostępna pod adresem: <http://localhost:3000>

6.6 Uruchomienie testów

```
cd api  
./mvnw test
```

Testcontainers automatycznie pobierze i uruchomi obraz PostgreSQL. Testy nie wymagają żadnej zewnętrznej bazy danych ani dodatkowej konfiguracji.

6.7 Dane seedowe

Przy pierwszym uruchomieniu backendu automatycznie tworzone są konta demonstracyjne:

Email	Hasło	Rola
admin@clearbook.com	Admin123!	Administrator platformy
patient@clearbook.com	Patient123!	Pacjent
doctor@clearbook.com	Doctor123!	Lekarz (profil: Anna Nowak, kardiologia)

Sprawozdanie dotyczy stanu repozytorium z dnia 9 czerwca 2026.